

Fila

Fila é um conceito fundamental na disciplina de Estruturas de Dados, que se refere a uma coleção linear de elementos em que a ordem de entrada e saída dos elementos é controlada de acordo com a regra "Primeiro a Entrar, Primeiro a Sair" (PEPS, do inglês, FIFO - *First In, First Out*). Isso significa que o primeiro elemento que entra na fila será o primeiro a sair dela.

Assim, trabalha-se com uma estrutura de dados que pode ser visualizada da seguinte forma:

início		fim
A	B	C

Sendo que ao realizar a inserção de um novo elemento na fila, seguindo a regra FIFO, ele somente poderá ser inserido após o último elemento, como visto abaixo, ao inserir o valor 'D'.

início			fim
A	B	C	D

Ainda seguindo a regra FIFO, ao remover um elemento da fila, só o poderemos fazer em seu início, removendo então o valor 'A', ficando a fila da seguinte forma:

início		fim
B	C	D

Assim, as principais características das filas são:

Inserção: a inserção de elementos ocorre no final da fila, também conhecido como "*tail*" ou "*cauda*". Quando um novo elemento é adicionado, ele se torna o último elemento da fila.

Remoção: a remoção de elementos acontece no início da fila, chamado de "*head*" ou "*cabeça*". O elemento que foi inserido há mais tempo e ainda não foi removido é sempre o primeiro a ser retirado.

Restrição de acesso: os elementos da fila não podem ser acessados diretamente, apenas por meio das operações de inserção e remoção. Isso torna a fila uma estrutura de dados restritiva em comparação com outras, como listas e *arrays*.

As filas podem ser implementadas de várias maneiras, como por meio de *arrays*, listas encadeadas, ou até mesmo usando duas pilhas. Algumas variações de filas são:

Fila estática: utiliza um *array* de tamanho fixo, o que limita a quantidade máxima de elementos na fila. Para lidar com a restrição de espaço, pode-se utilizar a técnica de "fila circular", em que os índices do *array* são tratados como circulares.

Fila dinâmica: usa uma lista encadeada ou um *array* dinâmico para armazenar os elementos, permitindo que a fila cresça e diminua de tamanho conforme necessário.

Fila de prioridade: nessa variação, os elementos são inseridos na fila com base em uma prioridade predefinida. A remoção ocorre de acordo com a prioridade, com o elemento de maior prioridade sendo removido primeiro.

As filas são amplamente utilizadas em diversos contextos, como em sistemas operacionais para gerenciar processos e em algoritmos de simulação de eventos discretos. Além disso, são fundamentais para abordar problemas envolvendo grafos e árvores, como em buscas em largura (BFS - *Breadth-First Search*).

Fila Estática

A fila estática é uma das maneiras de implementar a estrutura de dados conhecida como fila, que, como já citado, segue a regra "*First In, First Out*" (FIFO) – “Primeiro a Entrar, Primeiro a Sair” (PEPS). A fila estática utiliza um *array* de tamanho fixo para armazenar seus elementos. Devido à limitação de tamanho, é importante gerenciar o espaço do *array* de maneira eficiente para evitar a falta de espaço quando houver necessidade de armazenar mais elementos.

As funções básicas de uma fila são:

- Enfileirar (*enqueue*): adiciona um elemento ao final da fila. Na implementação de fila estática, isso significa inserir o elemento na próxima posição disponível do *array*.
- Desenfileirar (*dequeue*): remove o primeiro elemento da fila, ou seja, o elemento que está no início da fila. Na implementação de fila estática, envolve mover todos os outros elementos uma posição à frente no *array*. Essa abordagem, no entanto, pode ser ineficiente devido à realocação constante dos elementos.
- Ver o primeiro elemento: retorna o valor do primeiro elemento da fila.
- Verificar se a fila está vazia: Retorna verdadeiro se a fila estiver vazia, ou seja, se não houver elementos na fila. Na implementação de fila estática, isso pode ser feito comparando os ponteiros de início e fim da fila.

- Verificar se a fila está cheia: retorna verdadeiro se a fila estiver cheia, ou seja, se não houver espaço disponível para armazenar mais elementos. Na implementação de fila estática, isso pode ser feito verificando se a próxima posição disponível no *array* é igual ao tamanho do *array*.

A implementação de uma fila estática é semelhante à implementação de uma lista estática sequencial, com a diferença de que a lista estática não segue a regra FIFO e permite o acesso direto aos elementos. Ambas as implementações utilizam *arrays* de tamanho fixo, mas as operações de inserção e remoção são diferentes. Na lista estática, os elementos podem ser inseridos e removidos em qualquer posição, enquanto na fila estática, a inserção ocorre no final e a remoção no início da fila.

Ao implementar uma fila estática, nos concentraremos nas suas funções básicas, que incluem *inserir*, *remover*, *ver_primeiro_elemento*, *esta_vazia* e *esta_cheia*. Devido à sua relação próxima com a lista estática sequencial, a adaptaremos para atender aos requisitos de uma fila, removendo funções desnecessárias e modificando os nomes das funções relacionadas: *inserir_fim* se torna *inserir* e *remover_inicio* se transforma em *remover*.

As funções básicas implementadas para a fila estática serão, então:

- *inserir*: adiciona um elemento ao final da fila. Na implementação de fila estática, corresponde à função *inserir_fim* da lista estática sequencial, que insere o elemento na próxima posição disponível do *array*.
- *remover*: remove o primeiro elemento da fila, ou seja, o elemento que está no início da fila. Na implementação de fila estática, equivale à função *remover_inicio* da lista estática sequencial, que remove o elemento do início do *array* e move todos os outros elementos uma posição à frente.
- *ver_primeiro_elemento*: retorna o primeiro elemento da fila sem removê-lo, permitindo uma visualização do elemento que será removido na próxima operação de remoção.
- *esta_vazia*: retorna verdadeiro se a fila estiver vazia, ou seja, se não houver elementos na fila.
- *esta_cheia*: retorna verdadeiro se a fila estiver cheia, ou seja, se não houver espaço disponível para armazenar mais elementos.

Ao adaptar a lista estática sequencial para criar uma fila estática, garantimos que as operações de inserção e remoção estejam alinhadas com a regra FIFO, enquanto aproveitamos a eficiência

do armazenamento em *array* e a simplicidade da implementação da lista estática. Desta forma, temos a implementação abaixo para o TAD *Fila_estatica*

```
class Fila_estatica:
    def __init__(self, tamanho):
        self.tam_maximo = tamanho
        self.vetor = [None] * tamanho
        self.quant = 0

    def inserir(self, valor):
        self.vetor[self.quant] = valor
        self.quant += 1

    def remover(self):
        for i in range(1, self.quant):
            self.vetor[i - 1] = self.vetor[i]
        self.quant -= 1

    def esta_vazia(self):
        return self.quant == 0

    def esta_cheia(self):
        return self.quant == self.tam_maximo

    def ver_primeiro_elemento(self):
        return self.vetor[0]

    def show(self):
        for i in range(self.quant):
            print(self.vetor[i], end=' ')
        print()

    def tamanho_atual(self):
        return self.quant

    def capacidade(self):
        return self.tam_maximo
```

As funções `show`, `tamanho_atual` e `capacidade` foram mantidas na classe `Fila_estatica` porque, embora não sejam operações essenciais para o funcionamento de uma fila, elas fornecem informações úteis sobre a estrutura e ajudam na depuração e na visualização dos elementos da fila.

- `show`: permite exibir os elementos presentes na fila na ordem em que estão armazenados. Essa função é útil para visualizar o estado atual da fila e pode ser útil na depuração de problemas ou para verificar se as operações de inserção e remoção estão funcionando corretamente.
- `tamanho_atual`: retorna a quantidade de elementos atualmente presentes na fila. Essa informação pode ser útil em várias situações, como para verificar se a fila está próxima de sua capacidade máxima ou se há espaço suficiente para adicionar mais elementos.
- `capacidade`: retorna o tamanho máximo do `array` que armazena os elementos da fila. Essa informação é útil para entender a limitação de espaço da fila estática e pode ser usada em decisões de redimensionamento ou escolha de uma estrutura de dados diferente, como uma fila dinâmica, se necessário.

Fila Circular

A função `remove` na implementação da fila estática envolve mover todos os elementos restantes uma posição à frente no `array`. No entanto, essa abordagem pode ser ineficiente, principalmente quando o número de elementos na fila é grande. Uma solução para melhorar a eficiência é utilizar a técnica de "fila circular", que trata os índices do `array` de maneira circular, usando dois ponteiros (um para o início e outro para o final da fila) para gerenciar a inserção e remoção de elementos.

A implementação de uma fila circular se beneficia do uso do operador módulo (%) na linguagem de programação. Esse operador permite calcular a posição do `array` de forma circular, retornando o resto da divisão entre dois números. Em outras palavras, ao realizar a operação $(i + 1) \% \text{tam_maximo}$, o resultado é o resto da divisão de $i + 1$ por `tam_maximo`. Isso garante que o valor retornado estará sempre no intervalo de 0 a `tam_maximo - 1`, mantendo a estrutura circular da fila.

Com a fila circular, após inserções e remoções, a fila mantém sua estrutura circular sem a necessidade de realocar todos os elementos. Isso resulta em uma melhoria significativa na

eficiência do gerenciamento de inserções e remoções na fila, especialmente quando comparado à abordagem de fila estática.

Tal qual a implementação da lista estática sequencial serviu de base para a implementação das funções da fila estática, a implementação da lista estática circular será adequada para a fila estática circular. Assim, para adaptar o código da classe `Lescircular` para a classe `Fila_estatica_circular`, precisamos modificar as funções `inserir_fim` e `remover_inicio` para `inserir` e `remover`, além de remover as funções `inserir_inicio` e `remover_fim`, que não são relevantes para uma fila. A nova classe `Fila_estatica_circular` ficará assim:

```
class Fila_estatica_circular:
    def __init__(self, tamanho):
        self.tam_maximo = tamanho
        self.vetor = [None] * tamanho
        self.inicio = 0
        self.fim = 0
        self.quant = 0

    def inserir(self, valor):
        self.vetor[self.fim] = valor
        self.fim = (self.fim + 1) % self.tam_maximo
        self.quant += 1

    def remover(self):
        self.inicio = (self.inicio + 1) % self.tam_maximo
        self.quant -= 1

    def esta_vazia(self):
        return self.quant == 0

    def esta_cheia(self):
        return self.quant == self.tam_maximo

    def show(self):
        for i in range(self.quant):
            print(self.vetor[(self.inicio + i) % self.tam_maximo], end=' ')
        print()

    def ver_primeiro_elemento(self):
        return self.vetor[self.inicio]
```

A implementação da `Fila_estatica_circular` acima utiliza a técnica de fila circular, que melhora a eficiência das operações de inserção e remoção de elementos em comparação à implementação de uma fila estática tradicional. Isso ocorre porque a fila circular evita a realocação de todos os elementos ao remover um item da fila, o que pode ser uma operação custosa em termos de tempo de processamento.

Embora a implementação use um `array` estático, ela ainda é bastante versátil devido ao tratamento circular dos índices. Isso permite a implementação de uma fila com um tamanho fixo sem perder eficiência nas operações básicas.

A implementação da fila estática circular é um excelente exercício para desenvolver habilidades de programação e compreender como funcionam as estruturas de dados e algoritmos internamente.

Fila Dinâmica

A fila dinâmica também segue o princípio *First In, First Out* (FIFO), ou seja, o primeiro elemento a entrar na fila é o primeiro a sair. Diferentemente da fila estática, que usa um `array` de tamanho fixo para armazenar os elementos, a fila dinâmica utiliza a ideia de lista encadeada para armazenar os elementos. Neste caso, a fila dinâmica possui dois ponteiros principais: um apontando para o início da fila (onde ocorrem as remoções) e outro apontando para o fim da fila (onde ocorrem as inserções).

A implementação da fila dinâmica oferece algumas vantagens em relação à fila estática. Primeiramente, a fila dinâmica pode crescer e diminuir de tamanho conforme necessário, adaptando-se às necessidades do programa em tempo de execução. Isso elimina a necessidade de definir um tamanho máximo de antemão e evita o desperdício de memória. Além disso, as operações de inserção e remoção na fila dinâmica são geralmente mais eficientes do que na fila estática, pois não há necessidade de realocar elementos no `array`.

Na implementação da fila dinâmica podemos observar a sua proximidade com a lista dinâmica simplesmente encadeada. Ambas são estruturas de dados baseadas em nós encadeados sendo que a principal diferença entre elas reside nas operações permitidas e nas regras de acesso aos elementos. Assim, a fila dinâmica pode ser implementada usando uma lista dinâmica simplesmente encadeada como base. Nesse caso, a fila dinâmica aproveita a flexibilidade da lista dinâmica simplesmente encadeada para armazenar seus elementos e gerenciar a alocação dinâmica de memória. Porém, a fila dinâmica restringe as operações permitidas, seguindo a

regra FIFO, o que a torna mais especializada e adequada para situações específicas em que a ordem de processamento dos elementos é importante.

Desta forma, conseguimos facilmente chegar à implementação da fila dinâmica abaixo:

```
class No:
    def __init__(self, valor, proximo):
        self.info = valor
        self.prox = proximo

class Fila_dinamica:
    def __init__(self):
        self.prim = self.ult = None
        self.quant = 0

    def inserir(self, valor):
        if self.quant == 0:
            self.prim = self.ult = No(valor, None)
        else:
            self.ult.prox = self.ult = No(valor, None)
        self.quant += 1

    def remover(self):
        if self.quant == 1:
            self.prim = self.ult = None
        else:
            self.prim = self.prim.prox
        self.quant -= 1

    def esta_vazia(self):
        return self.quant == 0

    def tamanho_atual(self):
        return self.quant

    def ver_primeiro(self):
        return self.prim.info
```



```
def show(self):
    aux = self.prim
    while aux != None:
        print(aux.info, end=' ')
        aux = aux.prox
    print()
```

Assim, o código da classe `Fila_dinamica` foi gerado com base na classe `Ldse` (Lista dinâmica simplesmente encadeada) através das seguintes modificações:

A classe `Ldse` foi renomeada para `Fila_dinamica`; a função `inserir_fim` foi substituída pela função `inserir`, já que, em uma fila, os elementos são sempre inseridos no final; a função `remover_inicio` foi substituída pela função `remover`, já que, em uma fila, os elementos são sempre removidos do início; as funções que não são relevantes para uma fila foram removidas do código, já que não são aplicáveis à fila dinâmica; a função `show` foi mantida, pois pode ser útil para exibir os elementos da fila e as outras funções, como `esta_vazia`, `tamanho_atual` e `ver_primeiro`, foram mantidas sem alterações, pois são úteis para verificar o estado da fila e obter informações sobre ela.

E por que não implementar usando como base a lista dinâmica duplamente encadeada, tendo assim uma fila duplamente encadeada?

Bom, só reforçando o conceito, uma fila duplamente encadeada é uma estrutura de dados em que cada nó possui referências tanto para o próximo nó quanto para o nó anterior. No entanto, em uma fila, não há necessidade de manter referências aos nós anteriores, pois as operações fundamentais são a inserção no final e a remoção no início. Vamos analisar as principais operações em uma fila para entender por que uma fila duplamente encadeada não é necessária:

Inserir: na fila, os elementos são inseridos no final. Com uma fila simplesmente encadeada, é possível manter um ponteiro para o último elemento e inserir o novo elemento diretamente após ele, atualizando o ponteiro do último elemento para o novo nó. Portanto, não há necessidade de um ponteiro para o nó anterior.

Remover: a remoção de elementos ocorre no início da fila. Com uma fila simplesmente encadeada, é possível manter um ponteiro para o primeiro elemento e atualizar esse ponteiro para o próximo elemento quando um item é removido. Novamente, não há necessidade de um ponteiro para o nó anterior.

Acesso aos elementos: em uma fila, os elementos são acessados e processados na ordem em que foram inseridos (FIFO). Não há necessidade de navegar para trás na fila ou realizar operações complexas que envolvam nós anteriores.

Por essas razões, uma fila simplesmente encadeada é suficiente para implementar uma fila, e uma fila duplamente encadeada não é necessária. Além disso, o uso de uma fila duplamente encadeada pode aumentar a complexidade da implementação e o uso de memória, já que cada nó precisa armazenar ponteiros adicionais para os nós anteriores.

Fila de prioridades

Fila de Prioridades é uma estrutura de dados avançada que combina as características de uma fila com a capacidade de organizar seus elementos de acordo com a prioridade atribuída a eles. Assim como uma fila comum, a fila de prioridades permite a inserção e remoção de elementos, mas o elemento removido será sempre aquele com a maior prioridade.

Em uma fila de prioridades, os elementos são armazenados com um valor de prioridade associado, que determina sua posição na estrutura. Em geral, quanto maior a prioridade, mais próximo do início da fila estará o elemento. Quando um elemento é removido da fila, ele é sempre o elemento com a maior prioridade. Se houver elementos com prioridades iguais, a ordem de remoção seguirá a ordem de inserção, ou seja, o primeiro elemento inserido com a maior prioridade será removido primeiro.

As filas de prioridades são especialmente úteis em situações em que é necessário gerenciar tarefas ou eventos com diferentes níveis de importância. Alguns exemplos de aplicações incluem sistemas operacionais (para gerenciar a alocação de recursos a processos), algoritmos de otimização (como o algoritmo de Dijkstra para encontrar o caminho mais curto em um grafo), simulações de eventos discretos e sistemas de gerenciamento de tarefas.

Existem várias abordagens para implementá-la:

Inserção no final e remoção por prioridade: nesta abordagem, os elementos são inseridos no final da fila conforme chegam. Para remover um elemento, a fila é percorrida em busca do elemento de maior prioridade, que é então removido. Essa abordagem pode ser ineficiente em filas grandes, já que a busca pelo elemento de maior prioridade pode levar tempo linear em relação ao tamanho da fila.

Consideremos então um conjunto de prioridades numéricas, em que o maior número é a maior prioridade, e um conjunto de elementos que são adicionados à fila na seguinte sequência (tem-se o valor do elemento seguido de sua prioridade):

A – 1

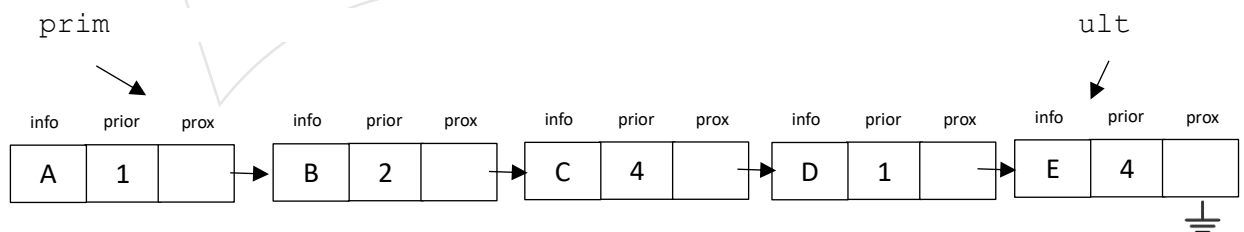
B – 2

C – 4

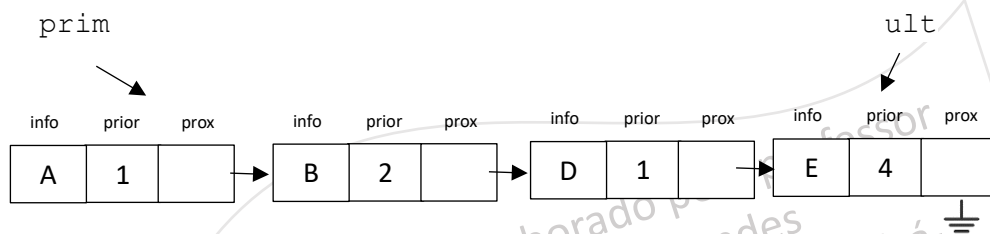
D – 1

E – 4

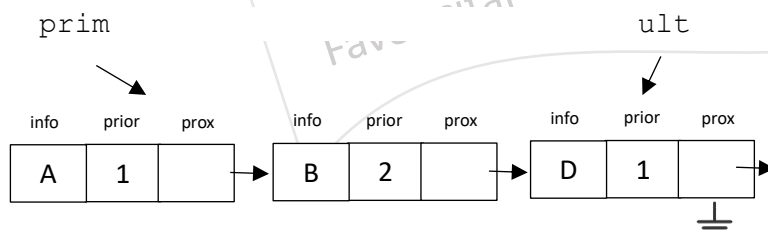
A fila seria formada pela inserção destes dados de forma sequencial, ficando com a seguinte configuração:



A executar a função remove será buscado o elemento com maior prioridade, no caso o elemento “C”, e seu nó será removido da fila.



Executando mais uma vez a função remove teríamos a fila da seguinte forma, pois o valor com maior prioridade é agora o valor “E”:



Assim, nesta abordagem teríamos a seguinte implementação:

```
class No:
    def __init__(self, valor, prioridade, proximo):
        self.info = valor
        self.prioridade = prioridade
        self.prox = proximo

class Fila_prioridade:
    def __init__(self):
        self.prim = self.ult = None
        self.quant = 0

    def inserir(self, valor, prioridade):
        if self.quant == 0:
            self.prim = self.ult = No(valor, prioridade, None)
        else:
            self.ult.prox = self.ult = No(valor, prioridade, None)
            self.quant += 1

    def remover(self):
        if self.quant == 1:
            self.prim = self.ult = None
        else:
            aux = maior = self.prim
            maior_prioridade = self.prim.prioridade
            ant = None

            while aux is not None:
                if aux.prioridade > maior_prioridade:
                    maior_prioridade = aux.prioridade
                    print(aux.info, maior_prioridade)
                    maior = aux
                    ant_maior = ant
                    ant = aux
                    aux = aux.prox

            if maior == self.prim:
                self.prim = self.prim.prox
            else:
                ant_maior.prox = maior.prox
                if maior == self.ult:
                    self.ult = ant_maior
                    self.ult.prox = None

            self.quant -= 1
```

```

def esta_vazia(self):
    return self.quant == 0

def tamanho_atual(self):
    return self.quant

def ver_primeiro(self):
    return self.prim.info

def show(self):
    aux = self.prim
    while aux != None:
        print('Valor: ', aux.info, ' - prioridade: ', aux.prioridade)
        aux = aux.prox
    print('\n')

```

Como pode-se ver, a função `inserir` na classe `Fila_prioridade` como implementada acima é responsável por adicionar um novo nó à fila, mantendo a ordem de chegada dos elementos. Ela recebe como parâmetros o valor e a prioridade do novo nó a ser inserido. A função cria um novo nó com os parâmetros recebidos e o insere sempre ao final da fila. Se a fila estiver vazia, o novo nó se torna o primeiro e o último elemento da fila. Caso contrário, o novo nó é inserido após o último elemento, e o ponteiro `ult` é atualizado para apontar para o novo nó.

A função `remover` é responsável por remover o elemento com a maior prioridade da fila. Se houver apenas um elemento na fila, ambos os ponteiros `prim` e `ult` são definidos como `None`. Caso contrário, a função percorre a fila procurando pelo elemento de maior prioridade. Quando o elemento de maior prioridade é encontrado, ele é removido da fila, e os ponteiros são ajustados de acordo. Se o elemento removido for o primeiro da fila, o ponteiro `prim` é atualizado para apontar para o segundo elemento. Se o elemento removido for o último da fila, o ponteiro `ult` é atualizado para apontar para o elemento anterior.

Inserção ordenada e remoção do primeiro: os elementos são inseridos na fila de forma ordenada, de acordo com suas prioridades. Dessa forma, o elemento de maior prioridade sempre estará no início da fila. A remoção é eficiente, mas a inserção pode ser lenta, pois é necessário encontrar a posição correta para cada novo elemento.

Utilizando o mesmo um conjunto de elementos com suas prioridades, e adicionando-os seguindo a sua ordem de precedência, teremos uma fila que irá se formando como ilustrado a seguir:

A – 1

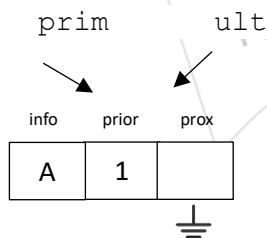
B – 2

C – 4

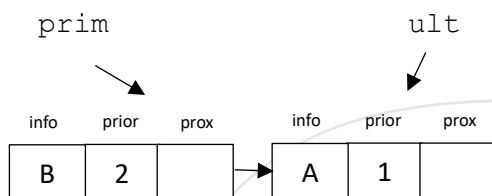
D – 1

E – 4

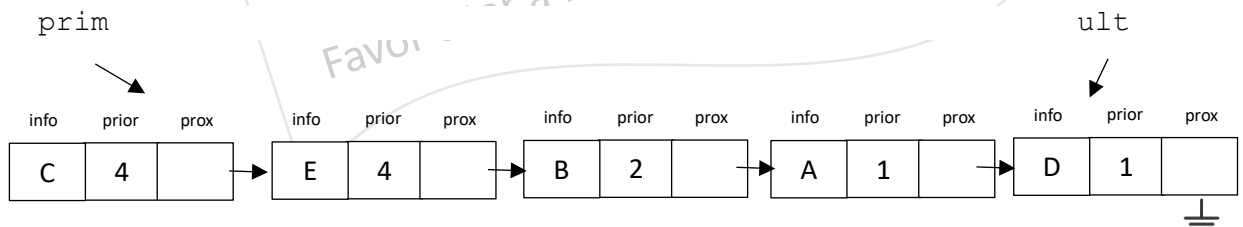
Inicialmente temos a fila com o primeiro elemento que recebemos de nossa relação acima:



Na sequência, inseriremos o elemento “B”, de prioridade 2, logo, com maior prioridade do que o elemento já inserido. A fila fica então da seguinte forma:

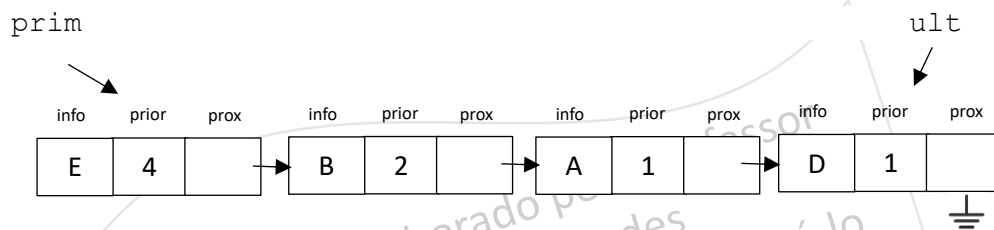


Fazendo as sucessivas inserções, teremos a fila com o seguinte desenho:



Vale a pena ressaltar que, na mesma prioridade, continua-se mantendo o critério de inserção FIFO, pois, como pode-se ver, o elemento “E”, que tem a mesma prioridade que o elemento “C”, é disposto após ele, logo, “E” somente será removido da fila após a remoção de “C”.

Pode-se ver isso ao remover um elemento da fila que, nesta forma de implementação, não precisará buscar pela fila inteira o valor com maior prioridade, pois este tem garantida sua posição no primeiro lugar da fila. Assim, ao executar a função remover para a fila acima, teríamos o seguinte resultado, com a remoção do nó com o elemento “C”:



Considerando agora esta abordagem teríamos a seguinte implementação:

```
class No:
    def __init__(self, valor, prioridade, proximo):
        self.info = valor
        self.prioridade = prioridade
        self.prox = proximo

class Fila_prioridade:
    def __init__(self):
        self.prim = self.ult = None
        self.quant = 0

    def inserir(self, valor, prioridade):
        if self.quant == 0:
            self.prim = self.ult = No(valor, prioridade, None)
        else:
            aux = self.prim
            ant = None

            while aux != None and aux.prioridade <= prioridade:
                ant = aux
                aux = aux.prox

            if aux == self.prim:
                self.prim = No(valor, prioridade, self.prim)
            elif aux == None:
                self.ult.prox = self.ult = No(valor, prioridade, None)
            else:
                ant.prox = No(valor, prioridade, aux)

            self.quant += 1

    def remover(self):
```

```

        if self.quant == 1:
            self.prim = self.ult = None
        else:
            self.prim = self.prim.prox
        self.quant -= 1

    def esta_vazia(self):
        return self.quant == 0

    def tamanho_atual(self):
        return self.quant

    def ver_primeiro(self):
        return self.prim.info

    def show(self):
        aux = self.prim
        while aux != None:
            print('Valor: ', aux.info, ' - prioridade: ', aux.prioridade)
            aux = aux.prox
        print()

```

Assim, mais uma vez, a função `inserir` na classe `Fila_prioridade` é responsável por adicionar um novo nó à fila, mantendo a ordem de prioridade. Ela recebe como parâmetros o valor e a prioridade do novo nó a ser inserido. A função percorre a fila para encontrar a posição correta para inseri-lo, de acordo com sua prioridade. Se a fila estiver vazia, o novo nó se torna o primeiro e o último elemento da fila. Se a prioridade do novo nó for maior que a prioridade do primeiro elemento da fila, o novo nó será inserido no início da fila. Se a prioridade do novo nó for menor ou igual à prioridade do último elemento da fila, o novo nó será inserido no final da fila. Caso contrário, o novo nó será inserido entre os elementos que têm prioridades maiores e menores que a sua.

A função `remover` é responsável por remover o elemento de maior prioridade da fila (que estará sempre no início da fila devido à ordem de inserção). Se a fila tiver apenas um elemento, os ponteiros `prim` e `ult` são definidos como `None`. Caso contrário, o ponteiro `prim` é atualizado para apontar para o segundo elemento da fila. Em ambos os casos, o atributo `quant` é decrementado em 1, representando a diminuição do tamanho da fila.

Filas separadas por prioridade: nesta abordagem, cria-se uma fila para cada nível de prioridade. Os elementos são inseridos na fila correspondente à sua prioridade. A remoção ocorre sempre

na fila de maior prioridade. Essa abordagem pode ser eficiente se houver um número limitado e bem distribuído de níveis de prioridade.

Vamos considerar o mesmo conjunto de elementos e suas prioridades:

A – 1

B – 2

C – 4

D – 1

E – 4

Inicialmente, com esta abordagem, teremos quatro filas vazias para cada prioridade:

fila1

prim = None



ult = None



fila2

prim = None



ult = None



fila3

prim = None



ult = None



fila4

prim = None

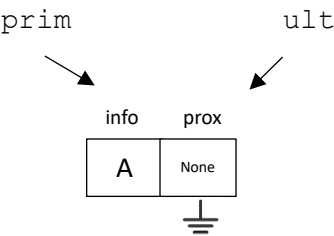


ult = None



Inserindo o elemento "A" com prioridade 1, teremos:

fila1



fila2

prim = None ult = None



fila3

prim = None ult = None



fila4

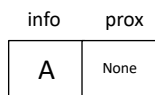
prim = None ult = None



Inserindo o elemento "B" com prioridade 2, teremos:

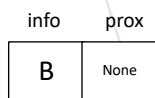
fila1

prim ult



fila2

prim ult



fila3

prim = None ult = None



fila4

prim = None



ult = None

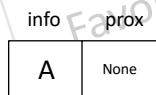


Inserindo o elemento "C" com prioridade 4, teremos:

fila1

prim

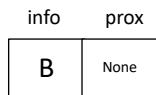
ult



fila2

prim

ult



fila3

prim = None



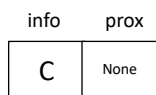
ult = None



fila4

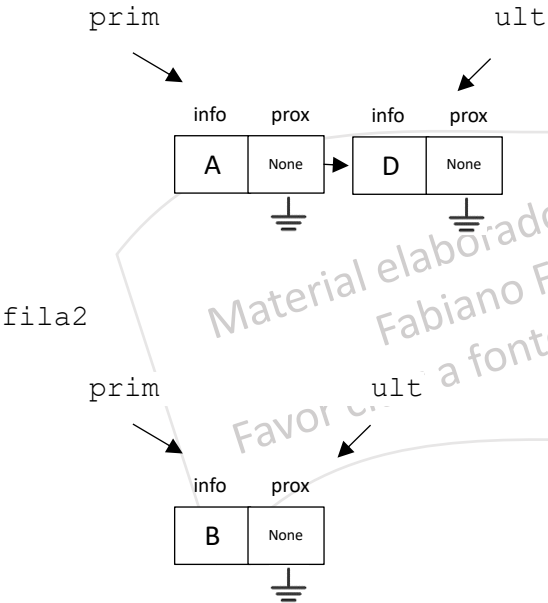
prim

ult



Inserindo o elemento "D" com prioridade 1, teremos:

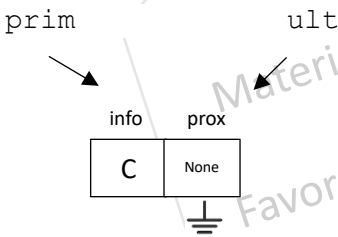
fila1



fila3

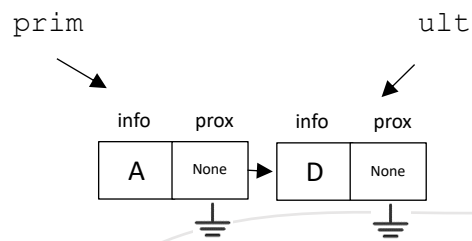
prim = None
ult = None

fila4

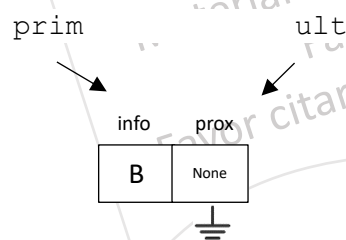


Inserindo o elemento "E" com prioridade 4, teremos:

fila1



fila2

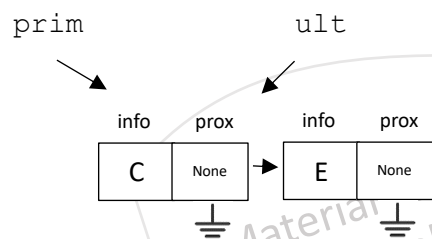


fila3

prim = None ult = None

```
graph LR; prim --- None1[None]; ult --- None2[None];
```

fila4

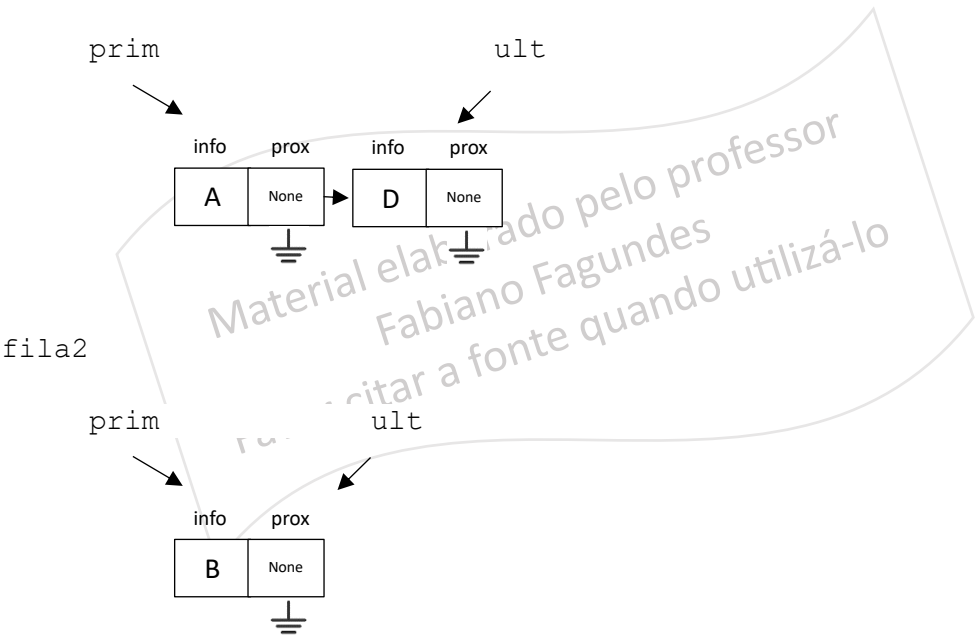


Observe que temos aqui uma fila3 vazia, pois não há nenhum elemento com prioridade 3. Nem por isso há desperdício de memória pois trata-se de uma fila dinâmica e não há espaço de memória alocado à espera de algum valor.

Também pode-se notar que não há a necessidade de guardar junto com o elemento sua prioridade, pois cada valor já armazenado na fila de sua respectiva prioridade.

Agora, ao remover um elemento da fila de prioridades, começamos verificando a fila de maior prioridade (no caso a fila criada para a prioridade 4) e removemos o primeiro elemento da fila. Neste caso, removemos o elemento "C" e teremos a fila da seguinte forma:

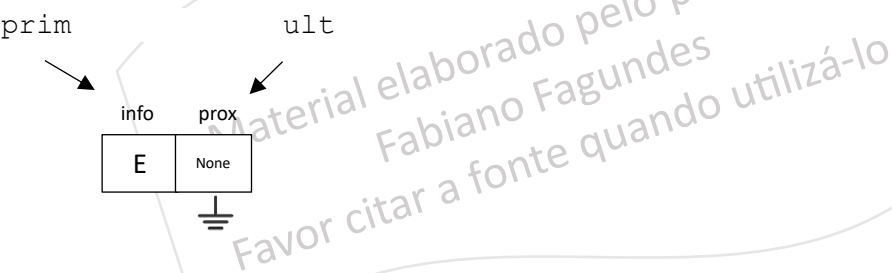
fila1



fila3

prim = None ult = None

fila4



Para esta configuração temos a implementação da fila de prioridades a seguir. Note que ela se utiliza da classe `Fila_dinâmica` já implementada anteriormente.

```
from fila_dinamica import fila_dinamica
class fila_prioridade:

    def __init__(self):
        self.fila1 = fila_dinamica()
        self.fila2 = fila_dinamica()
        self.fila3 = fila_dinamica()
        self.fila4 = fila_dinamica()

    def inserir(self, valor, prioridade):
        if prioridade == 4:
            self.fila4.inserir(valor)
        elif prioridade == 3:
            self.fila3.inserir(valor)
        elif prioridade == 2:
            self.fila2.inserir(valor)
        elif prioridade == 1:
            self.fila1.inserir(valor)

    def remover(self):
        if not self.fila4.esta_vazia():
            self.fila4.remover()
        elif not self.fila3.esta_vazia():
            self.fila3.remover()
        elif not self.fila2.esta_vazia():
            self.fila2.remover()
        elif not self.fila1.esta_vazia():
            self.fila1.remover()

    def esta_vazia(self):
        return self.fila0.esta_vazia() and self.fila1.esta_vazia() \
            and self.fila2.esta_vazia() and self.fila3.esta_vazia()

    def show(self):
        self.fila4.show()
        self.fila3.show()
        self.fila2.show()
        self.fila1.show()
```

A classe `fila_prioridade` conforme implementada acima utiliza quatro instâncias da classe `fila_dinamica`, uma para cada nível de prioridade (1 a 4, sendo esta última a maior prioridade).

Em seu construtor, agora, inicializa-se a fila de prioridades criando quatro filas dinâmicas (`fila1`, `fila2`, `fila3` e `fila4`), uma para cada nível de prioridade.

Na inserção, a função verifica a prioridade e insere o elemento na fila correspondente.

Já a função `remove` verifica primeiro se a fila de maior prioridade, a `fila4`, não está vazia. Se não estiver, ela remove o elemento dessa fila. Caso contrário, ela verifica a fila de prioridade 3 (`fila3`) e assim por diante, até encontrar uma fila que não esteja vazia e remover o elemento dela. Se todas as filas estiverem vazias, a função não faz nada.

A função `esta_vazia` aproveita-se do operador lógico `and`. Ela retorna `True` se todas as filas (`fila1`, `fila2`, `fila3` e `fila4`) estiverem vazias, e `False` caso contrário.

E, por fim, a função `show` imprime na tela os elementos e suas respectivas prioridades para cada fila percorrendo cada fila de prioridade. Para isso, ela chama a função `show` de cada instância da classe `fila_dinamica`, da fila de maior prioridade para a fila de menor prioridade.

Heap binário: um *heap* binário é uma árvore binária especial que segue uma propriedade específica (*min-heap* ou *max-heap*) e permite operações eficientes de inserção e remoção. Este modelo de implementação veremos quando entrarmos em árvores binárias, mais à frente.

Material elaborado pelo professor
Fabiano Fagundes
Favor citar a fonte quando utilizá-lo